



DELIVERY APP UPDATE · MARCH 29

Feature 4: Live Tracking

Bhavya

Feature Owner

29th March 2026

Table of Contents

Quick Recap: What We Had After March 28.....	3
File 1: TrackingController.java.....	3
File 2: tracking.html (Placeholder Template).....	4
What's Next.....	5

Quick Recap: What We Had After March 28

Before today, the entire backend of Feature 4 was already built and in place. A quick reminder of what existed:

- `DeliveryRequestRepository.java` — the database helper for orders. You write a method name like `findByStatus(PLACED)` and Spring generates `SELECT * FROM delivery_requests WHERE status = 'PLACED'` automatically. No SQL written by hand.
- `TrackingEventRepository.java` — fetches the full history of status changes for any order, sorted oldest to newest. This is the raw data that builds the tracking timeline.
- `TrackingService.java` — the brain of Feature 4. Runs the state machine (the rulebook for which status changes are allowed), saves `TrackingEvent` records, and tells the page which buttons to show agents.

Today we built the two files that connect all of that backend work to the browser.

File 1: `TrackingController.java`

A controller is the web layer of the app — it is the file that handles URLs. Think of it like a waiter: when someone makes a request (visits a URL or clicks a button), the controller takes that request, goes to the service to get what is needed, and returns the result. It does no thinking itself. All logic stays in the service.

Endpoint 1 — `GET /track/{orderId}`

When someone visits a URL like `/track/42` in their browser, this method runs:

1. Checks the session — is anyone logged in? If not, redirect to `/login`.
2. Fetches the order from the database using the ID from the URL.
3. Fetches the full tracking timeline (every status change ever recorded for this order).
4. Asks the service which status buttons are valid for this order right now.
5. Passes all of that data to the HTML template and returns the tracking page.

Endpoint 2 — `POST /track/{orderId}/update`

When an agent clicks a button like "Mark as Delivered", a form is submitted here:

1. Checks the user is logged in and is an AGENT or ADMIN — not a customer.
2. Converts the submitted text "DELIVERED" into the real Java enum value.
3. Calls `TrackingService` — which runs the state machine check, saves the `TrackingEvent`, and updates the order status in the database.
4. Redirects back to the tracking page (never returns HTML directly after a form submission).

The POST-Redirect-GET Pattern

After a form is submitted, the controller always redirects instead of returning HTML directly. If it returned HTML directly, refreshing the browser would re-submit the form — the status would change

twice by accident. By redirecting, a refresh just safely re-loads the page. This is a standard web pattern.

Session-Based Login Check

The controller uses `HttpSession` to check who is logged in. When someone logs in, their `User` object is stored in the session under the key `"loggedInUser"`. Every request checks that key. If it is null, nobody is logged in and they are sent to `/login`.

File 2: tracking.html (Placeholder Template)

Spring Boot needs a Thymeleaf HTML template file to exist in the templates folder for any page the controller tries to show. Without it, visiting `/track/42` would crash with an error. This file was created as a working placeholder so the app runs correctly while the final UI is decided later.

What Is Thymeleaf?

Thymeleaf is a templating engine. You write normal HTML, but add special `th:` attributes that Spring fills in with real data before sending the page to the browser. For example:

```
<h4 th:text="${order.id}">Order ID</h4>
```

Becomes in the browser:

```
<h4>42</h4>
```

What the Placeholder Template Currently Shows

- Order details — pickup address, delivery address, description, priority, time slot.
- Tracking timeline — every status change recorded for the order, with timestamps and agent notes.
- Agent action panel — buttons for valid next statuses. Only shown to agents and admins.
- Flash messages — success or error messages shown after a status update.

Will This Be Replaced?

Yes. The full UI will be designed and built at the end, once all backend features are complete. The controller does not care what the HTML looks like — it just passes the data. So the template can be fully redesigned without changing a single line of Java code.

What's Next

Feature 4 backend is fully complete. The remaining steps are:

Date	Task
March 30	Test the app — run it and check the full tracking flow works correctly
March 31	Fix any issues found in testing, final commit and push for Feature 4
April 1+	Begin Feature 5: Admin/Agent Dashboard (branch: feature-dashboard)
End of project	Bhavya designs and builds the full UI across all pages